

ECE 116: Advanced Hardware and Software Design

Intelligent Embedded Systems

Fall 2026 September 20, 2026

Instructor: Edward Wang, ejaywang@ucsd.edu

Meetings: Tuesday lecture on Zoom; Thursday lab/work in person, 9:30 a.m.-12:20 p.m. Pacific

Room: Jacobs Hall (EBU1), B538

Zoom: Use the class meeting link in Canvas.

Teaching assistant: Pameley Zantou

Edward's office hours: By appointment; email ejaywang@ucsd.edu to schedule. Use the class Zoom link.

Course communication: Canvas announcements are the official source for logistical updates

Course description

ECE116 is a hands-on course about building intelligent embedded systems. We begin with a deterministic device that senses the physical world and responds over time. We then add a provided voice model, train a vision model, deploy that vision capability to a constrained microcontroller, and apply a new on-device model to an application with a measured decision gate and useful end-action. Applications may operate locally or use connected services.

The course emphasizes engineering judgment. Students learn to distinguish a demonstration from evidence, measure behavior under documented conditions, identify failure at system boundaries, and defend a design under constraints such as latency, memory, energy, cost, reliability, security, and privacy.

No prior machine-learning experience is assumed. Machine learning is taught conceptually and through best practice. Mathematics is used when it helps make an engineering decision, not as a separate theory requirement.

Expected preparation

Students should arrive with introductory Python and basic Arduino experience associated with ECE18 or equivalent preparation. You should be comfortable with variables, conditionals, loops, functions, and running small programs. The course will teach the embedded architecture, evaluation, ML, and TinyML concepts required for A1-A4. Optional project resources support local actions, connected services, and application-specific integration.

Formal prerequisite language will follow the approved UC San Diego catalog and enrollment rules.

Learning outcomes

By the end of the course, you will be able to:

1. Organize sensing, timing, events, state, and output into responsive nonblocking embedded software.
2. Distinguish raw observations, model scores, decisions, and application events, then select decision policies based on stated consequences.

3. Design and interpret fair evaluations using appropriate data, metrics, latency, and resource measurements.
4. Build, train, and diagnose an image classifier in TensorFlow/Keras using split integrity, executable data pipelines, training evidence, error analysis, and domain-shift evaluation.
5. Carry a trained model through Edge Impulse BYOM to the XIAO, then verify preprocessing parity, memory, latency, accuracy, and throughput under device constraints.
6. Integrate a characterized on-device model, a justified decision gate, and a useful physical or software end-action.
7. Analyze how classification errors affect downstream system behavior and propose a mitigation supported by measured evidence.
8. Explain engineering choices using diagrams, measurements, limitations, a technical report, and a recorded demonstration.

How the course works

The two weekly meetings have different jobs:

- **Tuesday online lecture.** The three-hour block normally contains two related lecture modules, active reasoning, worked examples, and a break.
- **Thursday in-person lab or work.** A structured lab rehearses an essential transferable technique. A later Thursday is protected open work for design, integration, testing, and evidence, with course staff available.

The first meeting is Thursday, September 24, in person in B538. It includes L00 and L01 plus Lab 0: collect your kit, solder its headers with staff supervision, inspect the assembly, and verify basic board/toolchain operation. Bring your laptop. Lab 0 has no separate graded code submission. L02 and L03 meet on Zoom on Tuesday, September 29; the A1 structured lab remains Thursday, October 1.

The lecture and work strands are intentionally staggered. During an assignment's due week, Tuesday lectures begin the next conceptual topic while Thursday remains protected for the assignment being submitted. The next assignment's structured lab begins only after the current assignment has been submitted. All knowledge required for an assignment is taught by the end of the week before its deadline.

Structured labs are short, controlled exercises. They make you perform an important reasoning or implementation pattern before you apply it independently in the assignment challenge. Open work is not a new lab or an extra deliverable; it is supported time for design, integration, testing, and documentation.

Regular participation in both modes is expected. Thanksgiving removes the November 26 session. The final-project brief is available from the start of the quarter. November 17 contains a short project introduction and help time. November 24, December 1, and December 3 are supported project work. The project has no separate structured lab or proposal.

Course platform and materials

The course uses the Seeed Studio XIAO ESP32-S3 Sense throughout. The hardware path includes the onboard microphone and camera, plus a breadboard, light sensor, LEDs, resistors, and a pushbutton. Kits will be distributed during Lab 0 on Thursday, September 24. Staff will explain the kit inventory, assembly procedure, and how to report a missing or defective part.

You will need access to a computer that can run the pinned Arduino toolchain, Git, Python, and a modern browser. A3 uses a required Colab-hosted TensorFlow/Keras workflow for model development. A4 uses Edge Impulse BYOM for model import, conversion and profiling, Arduino library generation, and XIAO deployment. These form one supported sequence rather than two alternative training paths. Course-provided models and datasets will be released with the relevant assignments. Final project resources describe optional application directions; a paid cloud service is not required.

Bring the XIAO, a known-good USB data cable, and the components required by the current assignment to every Thursday in-person lab/work block that uses hardware.

The course textbook is provided as PDF chapters. Readings are assigned by lecture module. Assignment handouts describe the application and deliverables; textbook chapters and lectures teach the transferable ideas needed to complete them.

Assignment arc

A1: Deterministic Embedded Intelligence - 18.75 percent

Build a light-controlled timer using sensing, nonblocking timing, event interpretation, finite-state behavior, and visible output. The challenge is to turn noisy physical interaction into reliable state transitions without a button-driven script.

A2: Evaluate and Integrate a Provided Voice Model - 18.75 percent

Use a provided keyword model with real microphone input. Evaluate its predictions, choose and justify an operating point, turn repeated prediction windows into bounded command events, and integrate those events with the countdown timer implemented in A1.

A3: Train and Diagnose a Vision Classifier - 18.75 percent

Build and train a small gesture classifier in a Colab-hosted TensorFlow/Keras notebook. Preserve dataset provenance and split integrity, verify the executable input pipeline, make and explain transfer-learning choices, inspect training and class-level errors, compare a frozen model on a provided benchmark and protected self-collected deployment-domain set, and export the model with its input and label contract.

A4: Deploy and Choose Under Constraints - 18.75 percent

Continue the vision arc by evaluating frozen Keras-trained candidates, importing a selected artifact through Edge Impulse BYOM, generating and compiling its Arduino library, deploying it to the XIAO, verifying Keras-to-device preprocessing parity, measuring the device path, and defending a choice under a stated resource requirement.

Final project: Model-gated embedded application - 25 percent

Bring sensing, local machine learning, and application behavior together in an application you choose. Deploy a classifier for a new task with at least two target classes plus other/background. Make explicit acquisition/inference timing, threshold, and temporal event-policy decisions. Characterize the deployed classifier and justify the operating point using class-level results, ROC/AUC, precision, recall, and false-positive rates. Demonstrate a useful action and explain how model-output errors affect it. The 10-point failure criterion includes a proposed mitigation for a specific failure; an additional implemented version is not required.

Pretrained classifiers and coding assistance are allowed and encouraged. Kit sensors or self-acquired sensors may be used; no additional course sensor budget is provided. An office-hour meeting with Professor Wang to discuss your idea is strongly encouraged and optional. Email to arrange a time. There is no proposal.

The [final-project handout](#) defines the full requirements and rubric.

A1-A4 are self-contained. When an assignment depends on a prior capability, the course provides a known-good reference artifact. You may carry your own earlier work forward, but a problem or accommodation in one assignment will not prevent you from beginning the next. The final project may reuse infrastructure but requires a new model/task.

Assessment and grades

Component	Weight
A1: Deterministic embedded intelligence	18.75%
A2: Provided model evaluation and integration	18.75%
A3: Vision training and domain shift	18.75%
A4: TinyML deployment and selection	18.75%
Final project: Model-gated embedded application	25%
Total	100%

Assignment rubrics reward observable correctness, engineering reasoning, test quality, measurement, limitations, and reproducibility. Structured lab evidence is concise and checkable. The independent challenge carries most of each assignment's weight.

There is no separate participation grade. Lab checkoffs support learning and the assignment's required evidence; staff do not maintain participation scores.

Final project deliverables

Submit three files together to the Canvas final-project assignment:

- A report PDF containing four technical pages, one additional AI-use page, and one additional reference page. Figures and tables count within the technical pages; do not add a cover page or further appendices. Use the generic two-column Word template and choose your own headings and organization.
- A 3-5 minute MP4 demonstration showing the functioning device, sensor input, event gate, and useful action, including target, other/background, repeated-input, and challenging-input behavior. Captions or on-screen text convey essential narration.
- A trial-data ZIP with the real measurement records supporting the report and a description of their collection and interpretation. A separate source-code archive is not required.

Use `Final_Project_<student-id>.pdf`, `Final_Project_<student-id>.mp4`, and `Final_Project_<student-id>_Trial_Data.zip`. The report and data support analysis; the video shows functioning behavior. All contribute to one project grade.

The handout and template are available from the start of the quarter. Dedicated implementation follows A4's November 20 deadline. There is no required early deliverable.

Submission and deadlines

A1-A4 are submitted individually as one complete ZIP directly to Canvas. Include both Lab and Challenge work and the evidence required by the handout. No separate student repository or code submission is required. You may replace the complete ZIP before the hard close; staff grade the latest eligible submission, subject to an approved individual extension. Exact folder names, required artifacts, and acceptable file sizes are specified in each handout.

Course materials are published in a public GitHub repository linked from Canvas. Slides appear seven days before their lecture; assignment packages appear seven days before their assigned/start date. The final-project brief is available from the start of the quarter. Early access does not move a structured lab or deadline.

Assignments are due Fridays at 7:00 p.m. Pacific Time. Each assignment has an automatic grace period through 11:59 p.m. Friday and a single 72-hour hard submission cutoff:

Assignment	Assigned/start	Due	Canvas closes
A1	Thu Sep 24	Fri Oct 9, 7:00 p.m.	Mon Oct 12, 7:00 p.m.
A2	Fri Oct 9, after the A1 deadline	Fri Oct 23, 7:00 p.m.	Mon Oct 26, 7:00 p.m.
A3	Fri Oct 23, after the A2 deadline	Fri Nov 6, 7:00 p.m.	Mon Nov 9, 7:00 p.m.
A4	Fri Nov 6, after the A3 deadline	Fri Nov 20, 7:00 p.m.	Mon Nov 23, 7:00 p.m.
Final project	Brief from Week 0; dedicated work after A4, Nov 20	Fri Dec 4, 7:00 p.m.	Mon Dec 7, 7:00 p.m.

Late policy

Timing deductions are applied after grading; Canvas does not calculate the course's late penalty automatically. A Canvas late label during the grace period does not imply a deduction.

The grace period is automatic. You do not need to request or activate it. A submission timestamped by 11:59 p.m. Friday is treated as on time. A submission timestamped Saturday at 12:00 a.m. or later is officially late and receives a 25 percent deduction, equivalent to a maximum of 75 percent of the earned score.

Canvas closes the assignment 72 hours after the posted Friday 7:00 p.m. deadline, at 7:00 p.m. the following Monday. The grace period does not move this hard cutoff. After Canvas closes, the assignment cannot be submitted. There is no extension token or additional late tier. Documented accommodations and emergencies supersede this policy when arranged through the appropriate university process.

Collaboration and individual work

You are encouraged to discuss concepts, compare test strategies, help a classmate interpret an error message, and work near other students. Your submitted work must accurately identify your contribution and all external assistance. Measurements and hardware observations must come from your own work. The final project explicitly permits attributed external code and AI-generated implementation.

Do not exchange assignment code, completed notebooks, trained model artifacts, private solution repositories, or ready-to-submit prose. Do not use another student's measurements as your own. When you use external code, data, images, or technical sources, follow the assignment's permission rules and provide attribution.

Course staff may ask you to explain any part of a submission. Inability to explain submitted work may affect the score and may trigger an academic-integrity review.

Generative AI tools

Unless an assignment states otherwise, generative AI tools may be used as learning and debugging aids. For the final project, coding support is encouraged, including code generation, libraries, debugging, and refactoring. You remain responsible for the correctness, security, licensing, and explainability of everything you submit.

Substantive use must be disclosed in the assignment README (or final-project AI-use page) by naming the tool, describing what you asked it to help with, and explaining what you verified or changed. You may not use an AI tool to fabricate measurements, produce evidence for hardware you did not run, or invent sources.

An AI-generated answer is not evidence that a circuit works, a dataset is valid, a model generalizes, or a service is secure. Those claims require your own tests and reasoning.

Safety, privacy, and responsible data use

- Do not connect course hardware to mains voltage or equipment outside the released lab scope.
- Disconnect power before changing breadboard wiring when practical.
- Do not intentionally overdrive GPIO pins, LEDs, or other components.
- Obtain permission before recording another person's voice or image.
- Do not collect sensitive, identifying, or private images for course datasets.
- For remote processing, document the service and data flow, use authorized access, and keep credentials outside device code and submitted evidence. A paid service is not required.
- Never commit passwords, WiFi credentials, API keys, or tokens to a repository.
- Treat generated model output as untrusted data and keep physical actions behind deterministic application checks.
- Report exposed credentials, unsafe behavior, or privacy concerns to the instructor promptly.

Communication and support

Canvas announcements are the official source for schedule, release, and deadline changes. Use the course's published help channel for questions that can be discussed publicly. Send private matters involving grades, accommodations, health, or personal circumstances directly to the instructor through the designated Canvas or email channel.

The lecture-plus-work format is designed to provide frequent access to course staff. Bring a specific observation when requesting technical help: what you expected, what occurred, the smallest reproducible case, and the last known-good checkpoint.

Accessibility and inclusion

Students who need disability-related accommodations should contact the appropriate UC San Diego office and the instructor as early as possible. Course staff will work with approved accommodations while preserving the learning goals of the course.

Please also contact the instructor when hardware access, computing access, religious observance, caregiving, health, or another circumstance creates a barrier. Early communication usually provides more options.

The course uses microphones, cameras, visual demonstrations, serial output, and live hardware. Alternate access to required information will be provided when an approved accommodation or documented access need requires it.

Academic integrity

Follow the UC San Diego Policy on Integrity of Scholarship. Misrepresenting authorship, fabricating measurements, altering evidence to show behavior that did not occur, using an unauthorized solution, or sharing individual assignment work is not permitted.

You are always allowed to report that a system is incomplete or did not meet a requirement. Honest, well-documented failure can earn substantial partial credit. Fabricated success cannot.

Detailed course schedule

This weekly table is the calendar authority for course planning. When the schedule changes, update this table first and reconcile the master instructional map, current build state, assignment handouts, and onboarding guidance in the same revision. Changes affecting students will also be announced through Canvas.

Week	Dates	Tuesday: online lecture	Thursday: in-person work	Assignment milestones
Week 0	Sep 21-25	No class	Sep 24: L00 Class Introduction + L01 Programming Responsive Hardware; Lab 0 kit pickup, soldering, and bring-up	A1 starts Sep 24; final-project brief available
Week 1	Sep 28-Oct 2	Sep 29: L02 Reading Inputs and Controlling Outputs + L03 Responding to Events with State Machines	Oct 1: A1 structured lab	A1 instruction complete
Week 2	Oct 5-9	Oct 6: L04 Learning Patterns from Examples + L05 What Counts as Evidence	Oct 8: A1 open work	A1 due Oct 9 at 7:00 p.m.; A2 starts afterward
Week 3	Oct 12-16	Oct 13: L06 Classification Thresholds and Operating Points + L07 Streaming Predictions to Events	Oct 15: A2 structured lab	A1 closes Oct 12; A2 instruction complete
Week 4	Oct 19-23	Oct 20: L08 Neural Network Foundations + L09 How Neural Networks Learn	Oct 22: A2 open work	A2 due Oct 23 at 7:00 p.m.; A3 starts afterward
Week 5	Oct 26-30	Oct 27: L10 Convolutional Neural Networks + L11 Training CNNs for Generalization	Oct 29: A3 structured lab	A2 closes Oct 26; A3 instruction complete
Week 6	Nov 2-6	Nov 3: L12 Embedded Inference Resources + L13 Deployment and Preprocessing Contracts	Nov 5: A3 open work	A3 due Nov 6 at 7:00 p.m.; A4 starts afterward
Week 7	Nov 9-13	Nov 10: L14 Quantization and Model-Cost Tradeoffs + L15 Measuring and Choosing a Deployed System	Nov 12: A4 structured lab	A3 closes Nov 9; A4 instruction complete
Week 8	Nov 16-20	Nov 17: L16 Final Project Introduction; A4 help and project exploration	Nov 19: A4 open work	A4 due Nov 20 at 7:00 p.m.; final-project implementation starts afterward
Week 9	Nov 23-27	Nov 24: Final project open work (online)	Nov 26: No class, Thanksgiving	A4 closes Nov 23; Final project integration and testing
Week 10	Nov 30-Dec 4	Dec 1: Final project open work	Dec 3: Final project open work	Final project due Dec 4 at 7:00 p.m.